音影寻光 · 端侧障碍感知管线技术设想

本文是动笔前的设计底稿，用于固定管线分工、关键参数与实现顺序。文中帧耗时均为目标值，尚未在真机测量；未验证项已单独列出。

1. 核心判断：距离是安全信号，命名只是增强

原方案以 YOLO 为主、深度为辅。这带来一个结构性缺陷：YOLO 只能识别训练过的类别，而视障用户最需要的「前面多远有东西」反而取决于分类模型当场是否命中。台阶、玻璃门、悬挂物、路沿这些高频风险，恰恰是通用检测模型的弱项。

因此改为**距优先**。深度模型负责发现障碍并给出米制距离，YOLO 只负责给已经发现的障碍贴标签。识别成功则播报类别名，识别失败则播报「障碍物」，距离信息在任何情况下都不丢失。这样一来，YOLO 漏检不再等于漏报，安全兜底从概率性的分类结果转移到确定性的几何量。

需要说明的是，这个分工对深度侧提出了额外要求：深度阶段必须独立输出结构化的候选列表，而不是一张深度图，否则就没有可播报的对象。管线中的第三步正是为此存在。

2. 管线与帧预算

五个阶段，深度与语义分频运行，播报由状态机控制。下图标注各阶段的输入输出、帧预算与唯一的降级分支。

</> HTML 块 [该内容不支持导出查看]

3. 关键参数与取舍

下表的取值不是为了好看，每一条都对应一个具体的失败场景。

维度	取值	理由
深度频率	10 Hz，每 3 帧一次	深度变化远慢于语义，降频后把帧预算让给检测模型
深度输入尺寸	256×256	原生 518 在手机上过慢；助盲场景不需要逐像素精度
距离取值	候选区域内深度 10 分位	中位数代表物体中心，用户实际撞上的是最近边缘
走廊范围	近端 ±0.35 m，3 m 处 ±1.2 m	

		按水平视场约 65° 构建梯形；走廊外不播报，避免被路侧物体持续打扰
语义频率	3–5 Hz，全帧推理	候选数量不可控，逐候选裁剪推理会让帧率崩溃；全帧加标签缓存更稳
匹配条件	包围框重叠度 ⩾ 0.3 且深度差 ⩽ 0.5 m	只靠重叠度会把远处同名物体错配到近处候选上
播报触发	新目标、距离变化 > 0.5 m、风险等级上升	最小间隔 1.5 s，最多两条；否则每帧都在读，用户无法分辨
地面校正	画面下部 40% 做平面拟合	米制模型存在系统性尺度偏差，不校正播报的距离不可信

4. 三处必须单独处理的边界情况

- **地面校正失败。**反光地面、纯色墙面、正对大面玻璃时，平面拟合会失败。此时降级为相对深度加固定阈值，仍能播报「前方有东西」，但不给具体米数——宁可信息少，不可信息错。
- **台阶与跌落。**台阶在深度图上不是「低深度块」，而是相对拟合地面的残差。显著低于平面判为下行台阶或坑洞，高于平面且更近判为上行台阶。这条支路优先级高于普通障碍，因为它是最容易造成摔倒的场景。
- **动态目标。**行人、车辆、自行车的深度读数抖动明显。需要按轨迹做时间平滑并加迟滞，否则播报的距离会在相邻两帧之间来回跳变。

5. 尚未验证的三件事

1. **米制权重的场景适配。**Depth Anything V2 的米制权重分室内与室外两套，本项目主要场景在室内，需要确认室外光照下精度退化到什么程度，必要时做小样本微调。
2. **真机双模型并行耗时。**当前帧预算只是估算，两个模型 INT8 量化后同机并行的实际耗时、发热与耗电必须实测，这决定最终播报延迟是否可接受。
3. **走廊几何的个体差异。**眼镜相机的俯仰角与佩戴位置因人而异，固定梯形走廊可能需要一次佩戴标定，或改为运行时自适应估计。

6. 实现顺序

1. **桌面原型。**用离线视频跑通五级管线，标定走廊掩膜、地面校正参数与匹配阈值，把效果可视化出来逐帧核对。
2. **端侧移植。**参数冻结后再上 Android：CameraX 取帧，两个模型分线程并行，替换原有单一检测链路。
3. **接入播报。**对接现有语音播报管理组件，把「检测到某物」升级为「多远、什么方向、是什么」。

先在桌面上把算法调对再上机。手机端盲调阈值是最耗时的做法，也是这类项目常见的返工起点。

7. 在项目中的位置

该管线是客户端本地能力，不依赖云端服务，与 Java 业务后端、Python 导购服务构成三条互相独立的技术主线。它补上了此前客户端只有识别、没有距离的短板；补完之后，「端侧多模态感知」才真正成为可以和实习经历区分开的独立叙事，而不是又一个用了 Redis 和 JWT 的项目。

8. 后端与 AI Agent 技术构想

端侧管线解决的是安全问题，后端要补的不是又一层微服务，而是三段实习里完全没有的增量：**AI 工程化能力**——Agent 编排、RAG、评测观测。这一章固定架构取舍，避免后面又被 Spring AI、微服务、Redis 这些已经在实习里证明过的技术带偏。

8.1 双服务为什么这么拆

Java 侧保持模块化单体，不拆 SpringCloud 微服务；Python FastAPI 独立进程承载 Agent；Redis 作为基础设施静默使用，但不再作为叙事重点。

本项目选择	放弃的方案	理由
Java 模块化单体 + Spring Cloud Gateway	拆 Consul/Gateway/Feign 微服务	华迪实习已写透整套 SpringCloud，再堆一遍没有信息增量；单体对这个体量足够
Python FastAPI 独立进程	Java 单体里直接调 LLM / 上 Spring AI	LLM 调用是独立故障域，可单独扩缩容；与 Java 以 OpenAPI 契约解耦；新模型、rerank、评测工具在 Python 生态先落地
Redis 静默使用	主打 Redis 多级缓存 / 分布式锁 / 资源池	浩元实习已覆盖 Redis/Redisson，项目再写一遍是重复
Docker Compose 编排	Kubernetes	项目体量不需要，上了也讲不深，面试容易被问穿

8.2 Agent 运行时：LangGraph，不是裸 LangChain

两者不是竞品，是两层：**LangChain 是组件库**（模型客户端、消息类型、工具定义、检索抽象），**LangGraph 是状态图运行时**（节点、条件边、循环、checkpoint、断点续跑）。高层 agent API 底层就跑在 LangGraph 上。

本项目用 LangGraph 把导购对话显式建成状态机：意图识别 → 工具路由 → 多轮追问 → 降级话术；checkpoint 持久化到 PostgreSQL，对话中断或部署后可从断点恢复。Function Calling 封装五个业务

工具：搜商品、看详情、商品对比、加入购物车、客服 FAQ；工具失败有重试和话术降级，不让模型对着空结果硬编。

8.3 RAG：让模型对事实有据可查

管线固定为：文档加载 → 分块 → embedding → 向量加关键词混合检索（RRF 融合）→ BGE rerank 重排 → prompt 组装。目的不是炫技，是让库存、规格、价格这类事实从检索结果里取，而不是让模型凭记忆编。配套 golden set 测召回率与回答正确率，作为回归基线。

8.4 评测与可观测：多数学生作品的空白

自托管 Langfuse 记录全链路 trace：每次模型调用、每个工具的输入输出、token 成本和延迟。构建 30–50 条评测样例，跟踪三个指标——工具调用成功率、检索命中率、回答正确率。这一层是简历上能和普通"调了个 LLM 接口"作品拉开差距的地方。

8.5 Spring AI 为什么不选

Spring AI 2.0 的 ChatClient 加 advisor 链能做轻量问答和 RAG，但没有图编排引擎，复杂状态流要自建；Java 侧再上 AI 框架，与三段 Spring 实习同质化，也讲不出 LangGraph 那种 checkpoint、断点续跑的深度。标准口径：业务主链路保持 Spring Boot，Agent 因 Python 生态与独立故障域拆为 Python 服务；Spring AI 只作为面试常识了解，不落地。

8.6 明确不做的清单

- Kubernetes、自建推理引擎（vLLM）、模型微调训练——超出项目可讲深度。
- 多智能体框架（CrewAI、AutoGen）、A2A 协议——单 Agent 导购场景用不上。
- Dify、Coze 等无代码 Bot 平台——简历减分，不体现工程能力。
- 复杂长期记忆系统（Mem0、Zep）——行业仍不成熟，短期 checkpoint 足够。

8.7 落地顺序

1. **最小流式服务。**FastAPI + DeepSeek + SSE 跑通流式对话。
2. **手写工具。**不依赖框架先写 2–3 个 Function Calling 工具，理解参数校验与错误回传。
3. **LangGraph 状态机。**把对话重构成图，checkpoint 落 PostgreSQL。
4. **RAG 管线。**商品与 FAQ 入库，混合检索加 rerank 接入 Agent。
5. **评测观测。**接 Langfuse，建 golden set，跑通指标回归。
6. **部署工程化。**Docker Compose 一键编排，GitHub Actions 双端测试。